

PostgreSQL & SQL

Maîtriser les bases de données relationnelles

PostgreSQL	SQL	DDL	DML
------------	-----	-----	-----

Jointures	Index	Transactions	Fonctions
-----------	-------	--------------	-----------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Comprendre le modèle relationnel et ses fondements théoriques
- ✓ Écrire des requêtes SQL complexes : jointures, sous-requêtes, agrégats
- ✓ Concevoir un schéma de base de données normalisé et évolutif
- ✓ Créer et utiliser des index pour optimiser les performances
- ✓ Écrire des fonctions et des triggers en PL/pgSQL
- ✓ Maîtriser les transactions pour garantir l'intégrité des données

Table des matières

Chapitre 1 — Le modèle relationnel

- 1.1 Histoire et fondements
- 1.2 Tables, lignes, colonnes
- 1.3 Clés primaires et étrangères
- 1.4 Normalisation

Chapitre 2 — SQL fondamental — DDL et DML

- 2.1 CREATE TABLE et types de données
- 2.2 INSERT, UPDATE, DELETE
- 2.3 SELECT et filtrage
- 2.4 Tri, pagination, agrégats

Chapitre 3 — Requêtes avancées

- 3.1 Jointures : INNER, LEFT, RIGHT, FULL
- 3.2 Sous-requêtes et CTEs
- 3.3 Fonctions de fenêtre
- 3.4 JSONB dans PostgreSQL

Chapitre 4 — Conception de schéma

- 4.1 Choix des types de données
- 4.2 Contraintes et règles d'intégrité
- 4.3 Migrations versionnées
- 4.4 Soft delete

Chapitre 5 — Performance et programmabilité

- 5.1 Index et plans d'exécution
- 5.2 Transactions ACID
- 5.3 Fonctions PL/pgSQL

5.4 Triggers

Le modèle relationnel

1.1 Histoire et fondements

Le modèle relationnel a été proposé par Edgar F. Codd, chercheur chez IBM, dans son article fondateur de 1970 : 'A Relational Model of Data for Large Shared Data Banks'. Avant Codd, les bases de données étaient hiérarchiques ou réseaux, imposant aux développeurs de connaître la structure physique des données. Le modèle relationnel abstrait complètement la structure de stockage : on manipule des tables logiques sans se soucier de la façon dont les données sont organisées sur disque.

PostgreSQL (prononcé 'post-gress-Q-L') est né en 1986 à l'Université de Berkeley sous le nom POSTGRES. Il est aujourd'hui le SGBD open source le plus avancé du monde, avec un support exceptionnel pour les types de données complexes (JSONB, tableaux, types géographiques), les transactions ACID, la Row Level Security, et les fonctions stockées dans de nombreux langages.

1.2 Tables, lignes, colonnes

Une table (ou relation) est une collection de lignes (tuples) partageant la même structure. Chaque colonne (attribut) a un nom et un type de données. Chaque ligne est unique grâce à une clé primaire. La puissance du modèle relationnel vient des jointures : la capacité à combiner des données de plusieurs tables à partir de leurs relations.

Définition — NULL en SQL

NULL représente l'absence de valeur — pas zéro, pas une chaîne vide, mais une valeur inconnue. NULL a des comportements surprenants : `NULL = NULL` est FAUX (il faut utiliser `IS NULL`), et toute opération arithmétique avec NULL retourne NULL. Traiter NULL correctement est l'une des sources d'erreurs les plus fréquentes en SQL.

1.3 Clés primaires et étrangères

Une clé primaire (PRIMARY KEY) identifie de façon unique chaque ligne d'une table. Elle doit être non-nulle et unique. En pratique, on utilise soit un UUID (universally unique identifier — parfait pour les systèmes distribués) soit un entier auto-incrémenté (SERIAL ou BIGSERIAL en PostgreSQL). Une clé étrangère (FOREIGN KEY) est une colonne dont la valeur doit correspondre à une clé primaire dans une autre table — c'est le mécanisme qui établit les relations entre tables.

Définition d'un schéma avec clés étrangères

```
-- Exemple : schema multi-tenant simplifié  
  
CREATE TABLE tenants (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE products (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
name TEXT NOT NULL,
price INTEGER NOT NULL CHECK (price >= 0),
stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
deleted_at TIMESTAMPTZ, -- soft delete
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

1.4 Normalisation

La normalisation est le processus de structuration d'un schéma pour éliminer la redondance et prévenir les anomalies de mise à jour. Les trois premières formes normales (1NF, 2NF, 3NF) couvrent 95 % des besoins pratiques. 1NF : chaque cellule contient une valeur atomique (pas de listes ou tableaux). 2NF : chaque attribut non-clé dépend de la TOTALITÉ de la clé primaire. 3NF : chaque attribut non-clé dépend directement de la clé primaire (pas d'une autre colonne non-clé).

SQL fondamental — DDL et DML

2.1 CREATE TABLE et types de données

SQL se divise en plusieurs sous-langages. DDL (Data Definition Language) définit la structure : CREATE, ALTER, DROP. DML (Data Manipulation Language) manipule les données : INSERT, UPDATE, DELETE, SELECT. PostgreSQL offre une richesse de types de données bien au-delà des standards SQL.

Types de données PostgreSQL essentiels

Type PostgreSQL	Usage	Exemple
UUID	Identifiants universels	id UUID DEFAULT gen_random_uuid()
TEXT	Chaînes de longueur variable	name TEXT NOT NULL
INTEGER / BIGINT	Entiers (prix en centimes, quantités)	price INTEGER NOT NULL
BOOLEAN	Vrai/faux	is_active BOOLEAN DEFAULT true
TIMESTAMPTZ	Date+heure avec fuseau (toujours UTC)	created_at TIMESTAMPTZ DEFAULT NOW()
JSONB	JSON binaire indexable	metadata JSONB DEFAULT '{}'
TEXT[]	Tableau de chaînes	tags TEXT[] DEFAULT '{}'

2.2 INSERT, UPDATE, DELETE

```
-- INSERT simple
INSERT INTO products (tenant_id, name, price, stock_quantity)
VALUES ('abc-123', 'Chemise en soie', 15000, 50);

-- INSERT avec retour des valeurs créées
INSERT INTO orders (tenant_id, status, total_amount)
VALUES ('abc-123', 'new', 75000)
RETURNING id, created_at;

-- UPDATE avec condition
UPDATE products
SET stock_quantity = stock_quantity - 2,
    updated_at = NOW()
```

```
WHERE id = 'prod-456' AND tenant_id = 'abc-123'
AND stock_quantity >= 2; -- Vérification atomique

-- Soft DELETE (ne pas effacer physiquement)
UPDATE products
SET deleted_at = NOW()
WHERE id = 'prod-456' AND tenant_id = 'abc-123';

-- Hard DELETE (rare en production)
DELETE FROM products WHERE id = 'prod-456';
```

2.3 SELECT et filtrage

SELECT est la commande la plus riche et la plus utilisée de SQL. Maîtriser ses clauses dans le bon ordre d'évaluation est essentiel pour comprendre comment une requête se comporte : FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT/OFFSET.

```
-- Requête complète avec toutes les clauses
SELECT
p.id,
p.name,
p.price,
p.stock_quantity,
CASE
WHEN p.stock_quantity = 0 THEN 'out_of_stock'
WHEN p.stock_quantity < p.alert_threshold THEN 'low_stock'
ELSE 'in_stock'
END AS stock_status
FROM products p
WHERE p.tenant_id = 'abc-123'
AND p.deleted_at IS NULL -- Exclure les soft-deleted
AND p.price BETWEEN 5000 AND 50000
AND p.name ILIKE '%chemise%' -- Recherche insensible à la casse
ORDER BY p.created_at DESC
LIMIT 20 OFFSET 0; -- Pagination
```

2.4 Agrégats et GROUP BY

```
-- Statistiques de ventes par statut
```



```
SELECT
status,
COUNT(*) AS order_count,
SUM(total_amount) AS total_revenue,
AVG(total_amount)::INTEGER AS avg_order_value,
MIN(created_at) AS first_order,
MAX(created_at) AS last_order
FROM orders
WHERE tenant_id = 'abc-123'
AND created_at >= NOW() - INTERVAL '30 days'
GROUP BY status
HAVING COUNT(*) > 0
ORDER BY total_revenue DESC;
```

Requêtes avancées

3.1 Jointures

Les jointures combinent des lignes de plusieurs tables selon une condition. Comprendre la différence entre INNER JOIN, LEFT JOIN et leurs variantes est indispensable pour interroger des schémas relationnels sans perte ou duplication de données.

```
-- INNER JOIN : seulement les lignes qui matchent des deux côtés
SELECT o.id, o.status, p.name, oi.quantity
FROM orders o
INNER JOIN order_items oi ON oi.order_id = o.id
INNER JOIN products p ON p.id = oi.product_id
WHERE o.tenant_id = 'abc-123'
AND o.status = 'delivered';

-- LEFT JOIN : toutes les commandes, même sans items
SELECT o.id, COUNT(oi.id) AS item_count
FROM orders o
LEFT JOIN order_items oi ON oi.order_id = o.id
WHERE o.tenant_id = 'abc-123'
GROUP BY o.id;
```

3.2 CTEs (Common Table Expressions)

Les CTEs (WITH ... AS) permettent de nommer des sous-requêtes et de les réutiliser dans la requête principale. Elles améliorent considérablement la lisibilité des requêtes complexes et permettent des requêtes récursives.

```
-- CTE : top produits vendus du mois
WITH monthly_sales AS (
SELECT
oi.product_id,
SUM(oi.quantity) AS total_qty,
SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM order_items oi
```

```

JOIN orders o ON o.id = oi.order_id
WHERE o.tenant_id = 'abc-123'
AND o.status = 'delivered'
AND o.created_at >= date_trunc('month', NOW())
GROUP BY oi.product_id
)
SELECT p.name, ms.total_qty, ms.total_revenue
FROM monthly_sales ms
JOIN products p ON p.id = ms.product_id
ORDER BY ms.total_revenue DESC
LIMIT 10;

```

3.3 JSONB dans PostgreSQL

Le type JSONB (JSON Binaire) de PostgreSQL est une fonctionnalité puissante qui permet de stocker et d'interroger des données semi-structurées directement dans la base. Contrairement au type JSON, JSONB est stocké sous forme binaire décomposée, ce qui le rend indexable et plus rapide à interroger.

```

-- Accéder aux propriétés JSONB
SELECT
id,
metadata->>'phone_number_id' AS phone_number_id, -- texte
(metadata->'waba_id')::text AS waba_id,
metadata @> '{"platform": "whatsapp"}'::jsonb AS is_whatsapp
FROM social_connections
WHERE tenant_id = 'abc-123';

-- Mettre à jour une clé dans JSONB
UPDATE social_connections
SET metadata = metadata || '{"verified": true}'::jsonb
WHERE id = 'conn-123';

-- Index GIN pour les requêtes JSONB
CREATE INDEX idx_connections_metadata ON social_connections USING GIN (metadata);

```

Conception de schéma

4.1 Contraintes et règles d'intégrité

PostgreSQL offre plusieurs mécanismes pour enforcer les règles métier au niveau de la base de données. Les contraintes CHECK vérifient une condition à chaque INSERT/UPDATE. Les contraintes UNIQUE garantissent l'unicité d'une colonne ou combinaison de colonnes. DEFAULT définit une valeur par défaut. Ces contraintes constituent la dernière ligne de défense contre les données incohérentes — elles s'appliquent même si une migration manuelle ou un script bugué contourne votre code applicatif.

```
CREATE TABLE orders (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES tenants(id),  
  status TEXT NOT NULL DEFAULT 'new'  
  CHECK (status IN ('new', 'confirmed', 'processing', 'delivered', 'cancelled')),  
  total_amount INTEGER NOT NULL CHECK (total_amount >= 0),  
  customer_name TEXT NOT NULL,  
  deleted_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
-- Contrainte d'unicité composite  
ALTER TABLE social_connections  
ADD CONSTRAINT unique_active_connection  
UNIQUE (tenant_id, platform, external_account_id);
```

4.2 Migrations versionnées

Une migration SQL est un fichier qui décrit un changement de schéma de façon incrémentale et irréversible. Chaque migration est numérotée, appliquée une seule fois, et jamais modifiée après avoir été committée. Ce versionnement permet de reproduire exactement le schéma de production sur n'importe quel environnement.

Performance et programmabilité

5.1 Index et plans d'exécution

Un index est une structure de données auxiliaire qui accélère les recherches. Sans index, PostgreSQL doit lire chaque ligne de la table (sequential scan). Avec un index B-tree sur une colonne, la recherche est en $O(\log n)$. Le plan d'exécution (EXPLAIN ANALYZE) montre comment PostgreSQL exécute une requête — c'est l'outil de base pour le diagnostic de performance.

```
-- Index sur les colonnes fréquemment filtrées
CREATE INDEX idx_products_tenant ON products (tenant_id)
WHERE deleted_at IS NULL; -- Index partiel : seulement les produits actifs

CREATE INDEX idx_orders_tenant_status ON orders (tenant_id, status, created_at DESC);

CREATE INDEX idx_messages_conversation ON messages (conversation_id, created_at DESC);

-- Analyser une requête lente
EXPLAIN ANALYZE
SELECT * FROM products
WHERE tenant_id = 'abc-123' AND deleted_at IS NULL
ORDER BY created_at DESC LIMIT 20;

-- Lire le résultat : 'Index Scan' = bon ; 'Seq Scan' sur grande table = problème
```

5.2 Transactions ACID

ACID est l'acronyme des quatre propriétés garanties par les transactions PostgreSQL. Atomicité : toutes les opérations de la transaction réussissent ou toutes échouent. Cohérence : la base passe d'un état valide à un autre état valide. Isolation : les transactions concurrentes ne se voient pas mutuellement. Durabilité : une fois commitée, une transaction survit à un crash du serveur.

```
-- Transaction pour une commande + mise à jour du stock
BEGIN;

-- Créer la commande
INSERT INTO orders (tenant_id, status, total_amount)
VALUES ('abc-123', 'confirmed', 30000);

-- Décrémenter le stock avec vérification
```

```

UPDATE products
SET stock_quantity = stock_quantity - 2
WHERE id = 'prod-456'
AND stock_quantity >= 2; -- Garantie atomique

-- Si la mise à jour n'a pas touché de lignes, rollback
-- (géré côté applicatif en vérifiant rowcount)

COMMIT; -- ou ROLLBACK si erreur

```

5.3 Fonctions PL/pgSQL

PL/pgSQL est le langage procédural de PostgreSQL. Les fonctions stockées s'exécutent directement dans le moteur de base de données, proches des données. Elles sont idéales pour les opérations qui nécessitent des transactions atomiques ou des calculs qui seraient coûteux à faire côté applicatif.

Fonction PL/pgSQL avec verrouillage et graphe de transitions

```

-- Fonction atomique : transition de statut d'une commande
CREATE OR REPLACE FUNCTION transition_order_status(
p_order_id UUID,
p_tenant_id UUID,
p_new_status TEXT
) RETURNS orders AS $$
DECLARE
v_order orders;
v_allowed TEXT[];
BEGIN
-- Verrouiller la ligne pour éviter les race conditions
SELECT * INTO v_order FROM orders
WHERE id = p_order_id AND tenant_id = p_tenant_id
FOR UPDATE;

IF NOT FOUND THEN
RAISE EXCEPTION 'ORDER_NOT_FOUND';
END IF;

-- Graphe de transitions valides
v_allowed := CASE v_order.status
WHEN 'new' THEN ARRAY['confirmed','cancelled']
WHEN 'confirmed' THEN ARRAY['processing','cancelled']
WHEN 'processing' THEN ARRAY['delivered','cancelled']

```

```

ELSE ARRAY[]::TEXT[]
END;

IF NOT (p_new_status = ANY(v_allowed)) THEN
RAISE EXCEPTION 'INVALID_TRANSITION: % -> %',
v_order.status, p_new_status;
END IF;

UPDATE orders SET status = p_new_status, updated_at = NOW()
WHERE id = p_order_id RETURNING * INTO v_order;

RETURN v_order;
END;

$$ LANGUAGE plpgsql SECURITY DEFINER;

```

■ À retenir

- ✓ Le modèle relationnel organise les données en tables liées par des clés étrangères
- ✓ Toujours utiliser des UUIDs comme clés primaires dans les systèmes distribués
- ✓ NULL ≠ zéro ≠ chaîne vide — traitement spécial avec IS NULL
- ✓ Les index B-tree accélèrent les colonnes souvent filtrées — créer des index partiels quand possible
- ✓ Les transactions ACID garantissent l'intégrité lors des opérations multi-tables
- ✓ PL/pgSQL exécute la logique directement dans le moteur — idéal pour les opérations atomiques critiques

— ■ Exercices

Exercice 1 — Schéma e-commerce

Concevoir et implémenter un schéma de base de données pour une boutique en ligne.

- Tables : customers, categories, products, orders, order_items, reviews
- Toutes les clés primaires en UUID, contraintes CHECK sur les statuts
- Créer au moins 5 index pertinents pour les requêtes typiques
- Insérer des données de test et vérifier les contraintes avec des INSERTs invalides

Exercice 2 — Rapport de ventes

Écrire une requête SQL qui génère un rapport complet des ventes du mois.

- Ventes totales par catégorie de produit
- Top 5 produits par chiffre d'affaires
- Évolution journalière du nombre de commandes
- Utiliser des CTEs pour structurer la requête

Exercice 3 — Fonction atomique

Créer une fonction PL/pgSQL pour annuler une commande avec remboursement du stock.

- Vérifier que la commande existe et appartient au bon tenant
- Interdire l'annulation si status = 'delivered'
- Remettre le stock si le statut était 'processing' ou au-delà
- Loguer l'action dans une table audit_log

Bibliographie & Ressources

Documentation officielle

- postgresql.org/docs — documentation PostgreSQL (très complète)
- postgresqltutorial.com — tutoriels pratiques avec exemples

Livres

- Craig Kerstiens — PostgreSQL (The Pragmatic Bookshelf)
- Hans-Jürgen Schöning — Mastering PostgreSQL 15 (Packt, 2023)

Outils

- pgAdmin — interface graphique officielle
- DBeaver — client SQL universel
- explain.dalibo.com — visualiseur de plans d'exécution EXPLAIN